

PROJECT DOCUMENTATION

AI FAQ ASSISTANT API

1. INTRODUCTION

- PROJECT TITLE: AI FAQ ASSISTANT
- TEAM LEADER:

NAME : RENUKAPRIYA.S

MAIL ID : priyarenuka060@gmail.com

NM ID : AF3799FB717DB9B900B404BF035E37BC

- TEAM MEMBERS :

NAME : RAJAKUMARI.B

MAIL ID : rajakumarib971@gmail.com

NM ID : 851CCEDB80D4C1E98D145037FC660343

NAME : RAKSHAYA.S

MAIL ID : s.rakshaya08@gmail.com

NM ID : 4D4B7DBF6F004CC82AE4F014F9E5B7EC

NAME : RAMYA.R

MAIL ID : ramya.r37161@gmail.com

NM ID : 1B1590BC5D30E1B9A07470FCFA309D1C

NAME : RINITHASRI C.H

MAIL ID : rinithasri577@gmail.com

NM ID : AF3799FB717DB9B900B404BF035E37BC

2. PROJECT TITLE

AI FAQ Assistant API – AI-powered FAQ and Customer Support Automation

3. PROJECT OVERVIEW

AI FAQ Assistant is a backend application that helps users create, manage, search, and generate frequently asked questions using AI. It provides secure authentication, CRUD operations, category management, semantic search, and AI-assisted FAQ generation.

KEY OBJECTIVES

- Provide secure user authentication using JWT
- Generate FAQ questions and answers using Google Gemini AI
- Enable Create, Read, Update, and Delete operations for FAQs
- Provide keyword-based FAQ search
- Organize FAQs into categories
- Store FAQ and user data securely in MongoDB
- Provide centralized error handling and data validation

TARGET USERS

- Administrators
- Content creators
- Authenticated users
- Public users
- Developers learning Node.js and AI integration

PROJECT INSTRUCTION

AI FAQ Assistant is a backend REST API project developed using Node.js and Express.js.

QUICK START:

```
npm init -y
```

```
npm install express mongoose dotenv cors jsonwebtoken bcrypt @google/genai
```

```
npm run dev
```

The application can be tested using Postman or Thunder Client.

4. PRE-REQUISITES

SOFTWARE REQUIREMENTS

Software	Purpose
Node.js	Backend JavaScript runtime
MongoDB	NoSQL database
Mongoose	Database modelling and validation
npm	Package management
Express.js	REST API framework
Postman / Thunder Client	API testing
VS Code	Code editor

HARDWARE REQUIREMENTS

- Intel Core i5 / AMD Ryzen 5 or equivalent
- Minimum 8 GB RAM
- Minimum 1 GB available storage

5. PROJECT ARCHITECTURE

The application follows a layered backend architecture. Requests are received through the Express server, processed by authentication middleware and controllers, and then connected to MongoDB through Mongoose. Google Gemini AI is integrated for AI-based FAQ generation.

MAIN COMPONENTS:

- Express Server
- Authentication Middleware
- Controllers / Logic Modules
- Mongoose Database Layer
- MongoDB Database
- Google Gemini AI Integration

6. DATABASE ARCHITECTURE

The system contains User, FAQ, AI Generation, and Category entities.

User Collection:

- `_id`
- Name
- Email
- Password
- Role
- CreatedAt
- UpdatedAt

FAQ Collection:

- `_id`
- Question
- Answer
- Category
- CreatedBy
- CreatedAt
- UpdatedAt

A registered user can create and manage multiple FAQ records. Categories can also contain multiple FAQ records.

7. FEATURES

Feature	Description
User Authentication	Secure JWT-based authentication
AI FAQ Generation	Generates FAQ questions and answers using Google Gemini
FAQ Search	Searches FAQs using keywords
CRUD Operations	Create, Read, Update and Delete FAQs
Category Management	Organizes FAQs into categories
Password Encryption	Uses bcrypt for password hashing
Error Handling	Provides centralized error handling

Schema Validation Validates data using Mongoose
MongoDB Integration Stores application data in MongoDB

These features are documented in the original AI FAQ Assistant specification.

8. ROLES AND RESPONSIBILITIES

Admin:

- Manage registered users
- View, update, and delete FAQs
- Manage FAQ categories
- Monitor AI-generated content

Content Creator:

- Create FAQs
- Edit personal FAQs
- Delete owned FAQs
- Generate FAQs using AI
- Organize FAQs into categories

Authenticated User:

- View FAQs
- Search FAQs
- Generate AI-assisted answers
- Access personal profile information

Public User:

- View published FAQs
- Search FAQs
- Cannot create, edit, or delete FAQs
- Cannot access protected APIs

9. PROJECT SETUP AND CONFIGURATION

Project Setup Steps:

1. Create a new project folder.
2. Create a src folder inside the project.
3. Open the project folder in VS Code.
4. Initialize the Node.js project.
5. Install the required packages.

6. Create the required folders and files.

Required Packages:

- Express
- Mongoose
- Dotenv
- Cors
- Jsonwebtoken
- Bcrypt
- Google GenAI

10. BACKEND DEVELOPMENT

- Setup the Express server
- Create server.js and app.js
- Configure environment variables
- Configure CORS and JSON middleware
- Implement JWT authentication
- Create authentication routes
- Create FAQ CRUD routes
- Create AI generation routes
- Implement authentication middleware
- Implement centralized error handling

11. DATABASE DEVELOPMENT

MongoDB is used as the primary database and Mongoose is used for schema definition and database operations.

User Model:

- Name
- Email
- Password
- Role

FAQ Model:

- Question
- Answer
- Category

- CreatedBy
- Timestamps

Mongoose validates the data before storing it in MongoDB.

12. API ROUTES

Authentication:

- POST /api/auth/register
- POST /api/auth/login

FAQ:

- POST /api/faqs
- GET /api/faqs/search?q=configure

AI:

- POST /api/ai/generate-faq

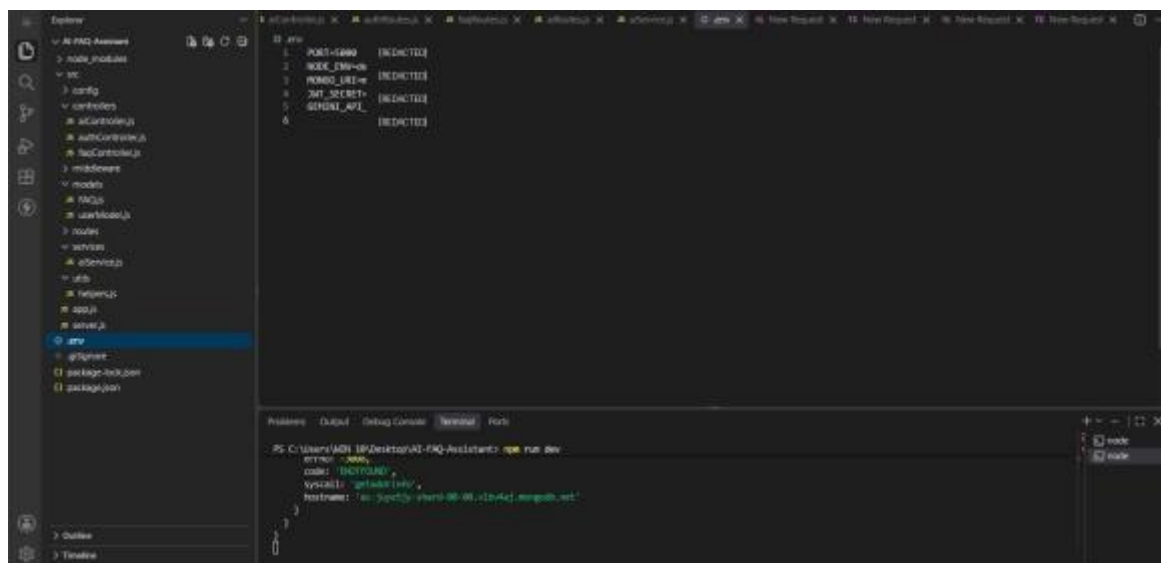
The private routes use JWT Bearer-token authentication.

13. PROJECT EXECUTION

Project execution can be performed using the following steps:

1. Install all required dependencies.
2. Configure MongoDB connection.
3. Start the backend server using `npm run dev`.
4. Open Postman or Thunder Client.

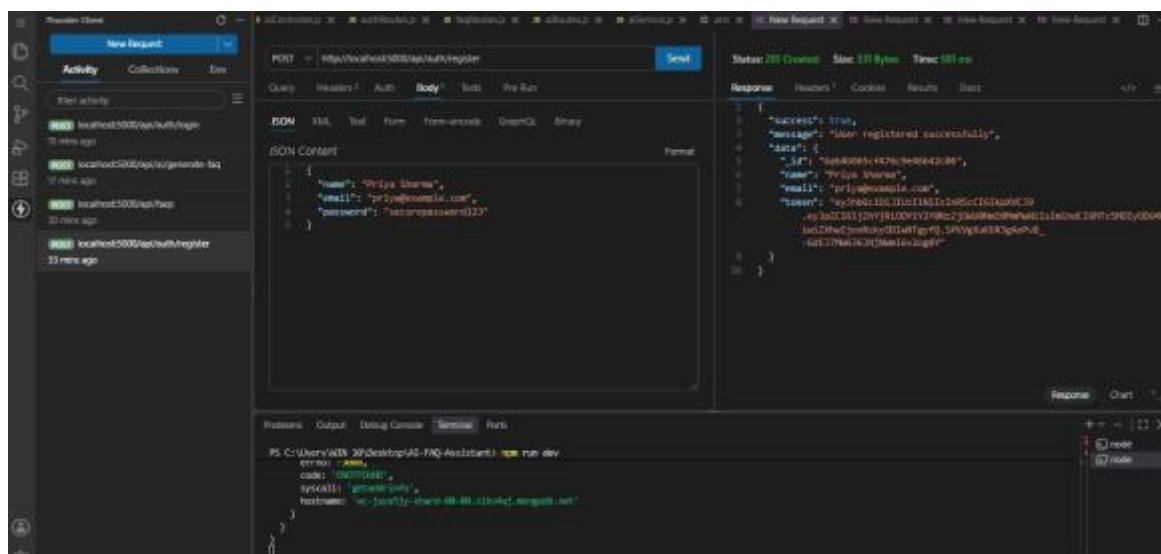
Figure 1. Environment Configuration (.env) – Sensitive Values Redacted



API testing evidence captured using Thunder Client.

5. Test the Register API.

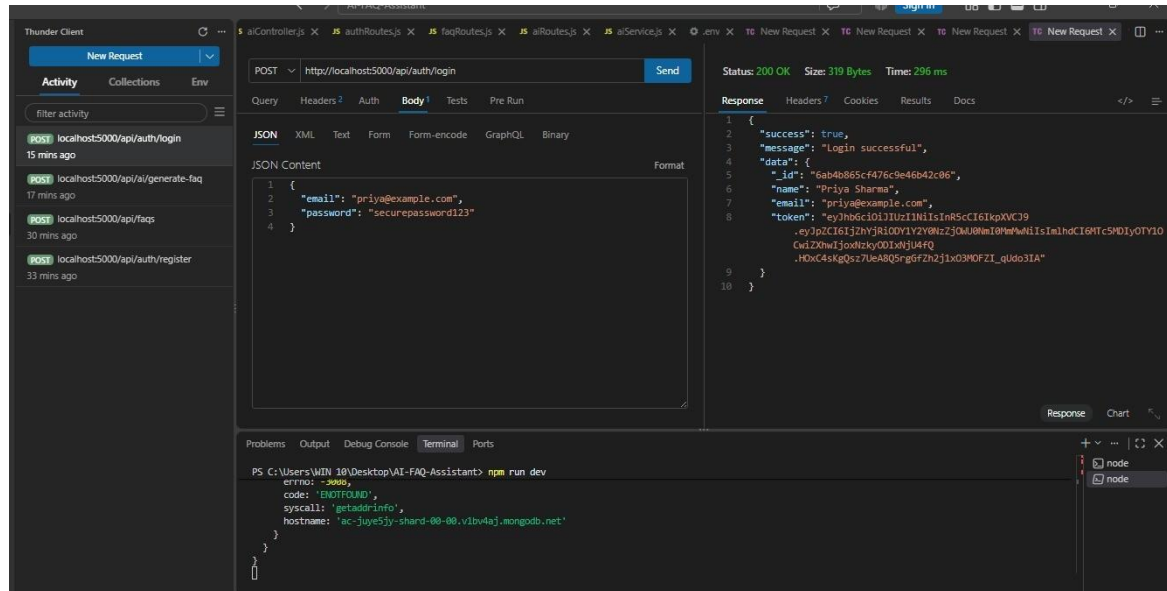
Figure 1A. User Registration – POST /api/auth/register



API testing evidence captured using Thunder Client.

6. Test the Login API.

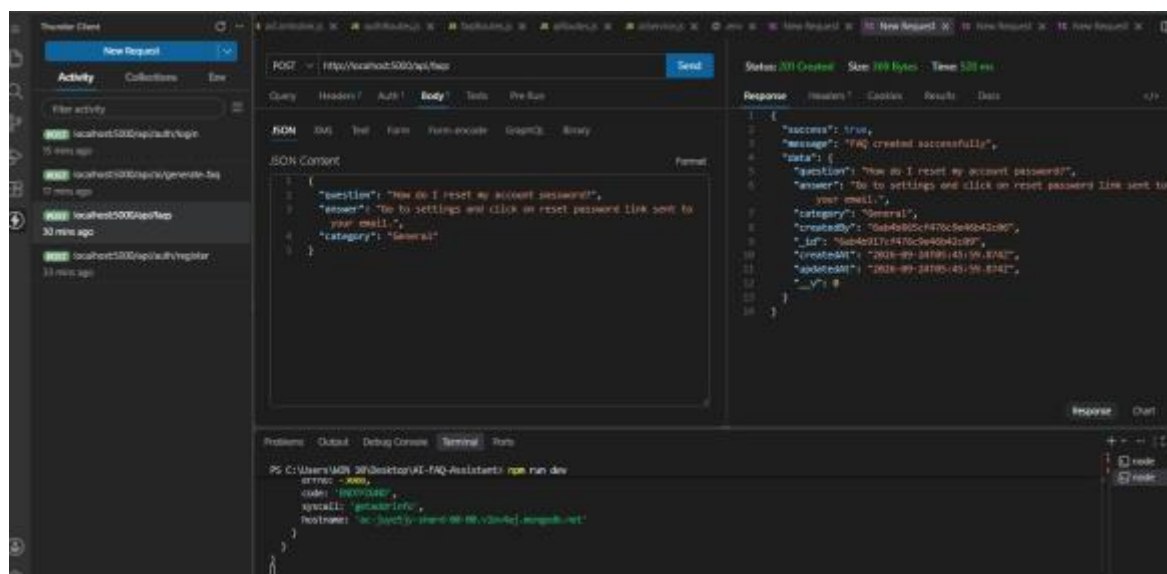
Figure 2. User Login and JWT Authentication – POST /api/auth/login



API testing evidence captured using Thunder Client.

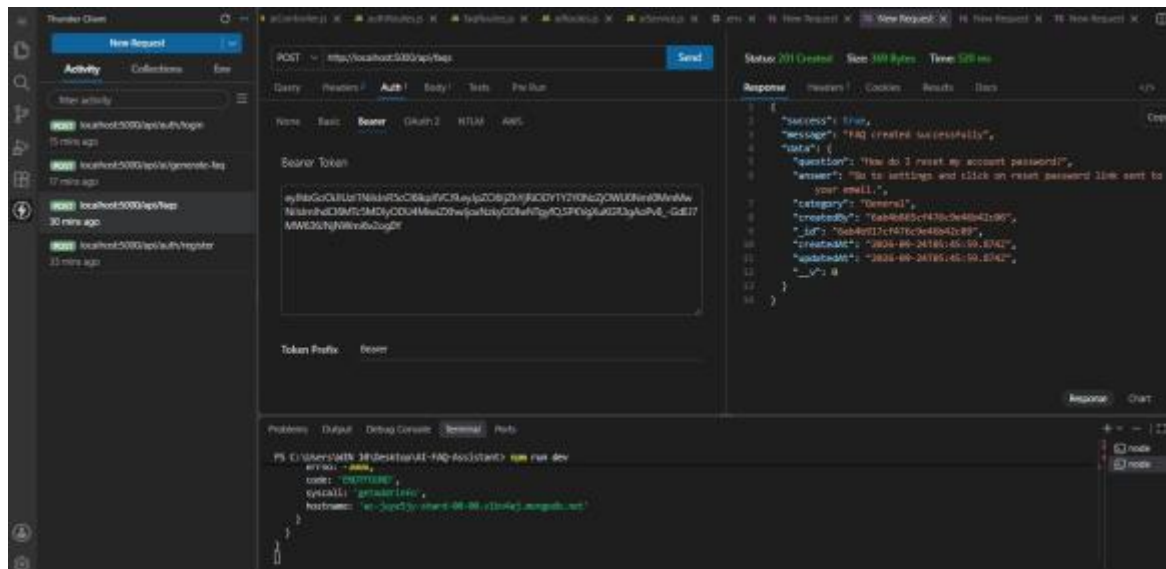
7. Obtain the JWT token.
8. Use the token to access protected APIs.
9. Create and manage FAQ records.

Figure 3. FAQ Creation Request and Response – POST /api/faqs



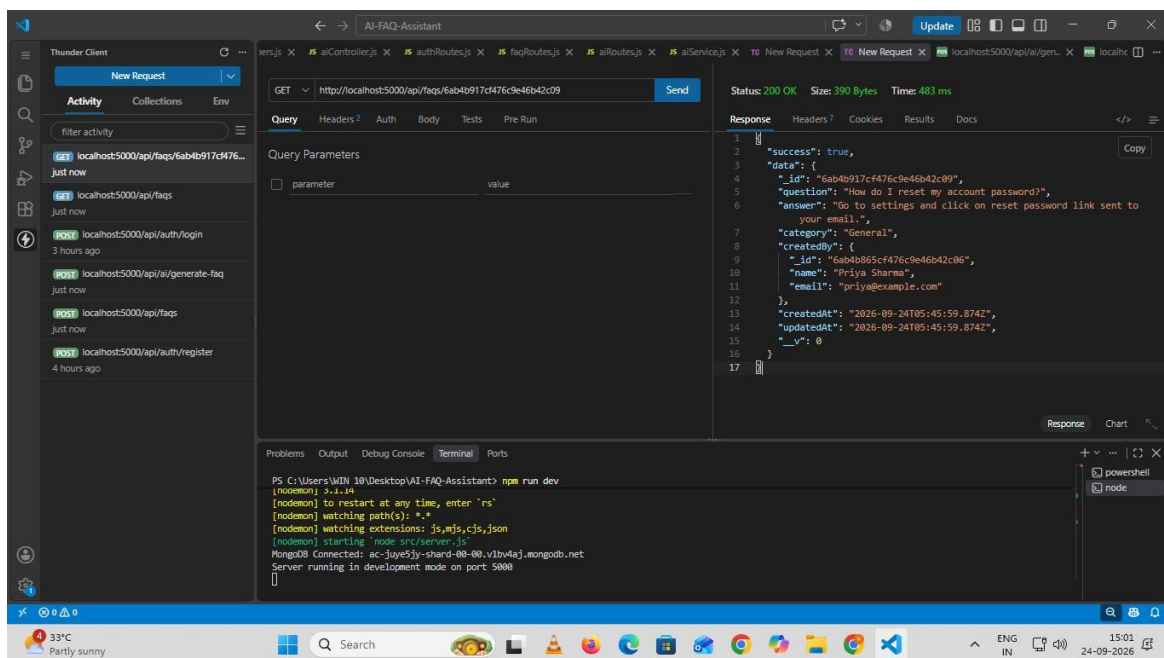
API testing evidence captured using Thunder Client.

Figure 4. FAQ Creation with JWT Authentication – POST /api/faqs



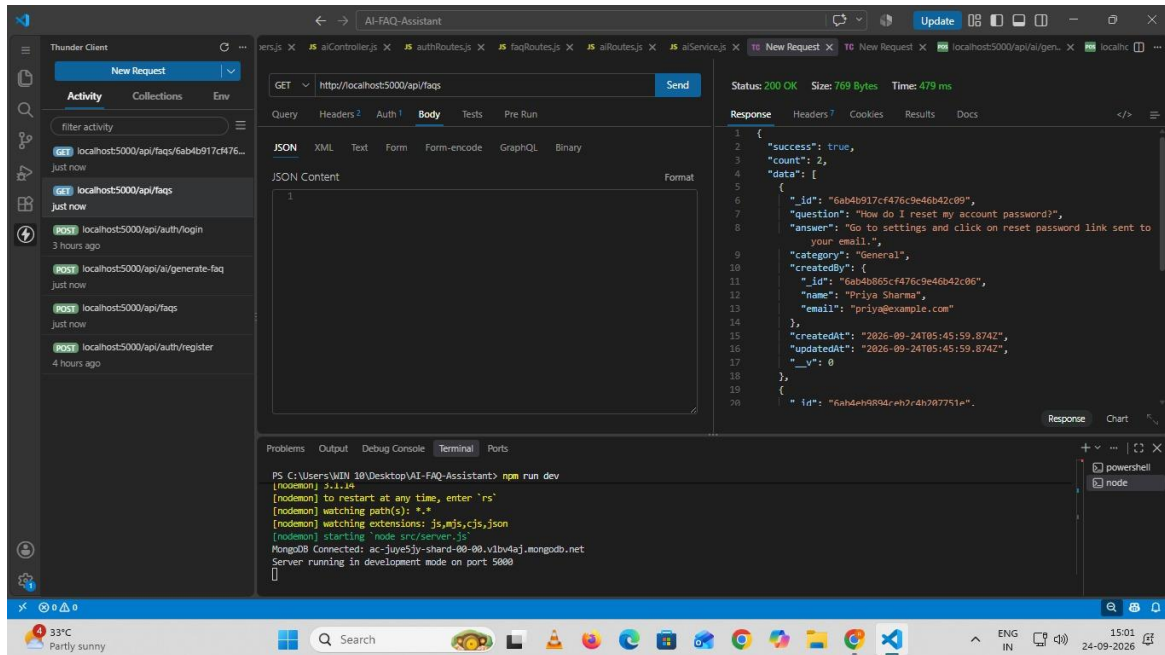
API testing evidence captured using Thunder Client.

Figure 5. FAQ Retrieval by ID – GET /api/faqs/:id



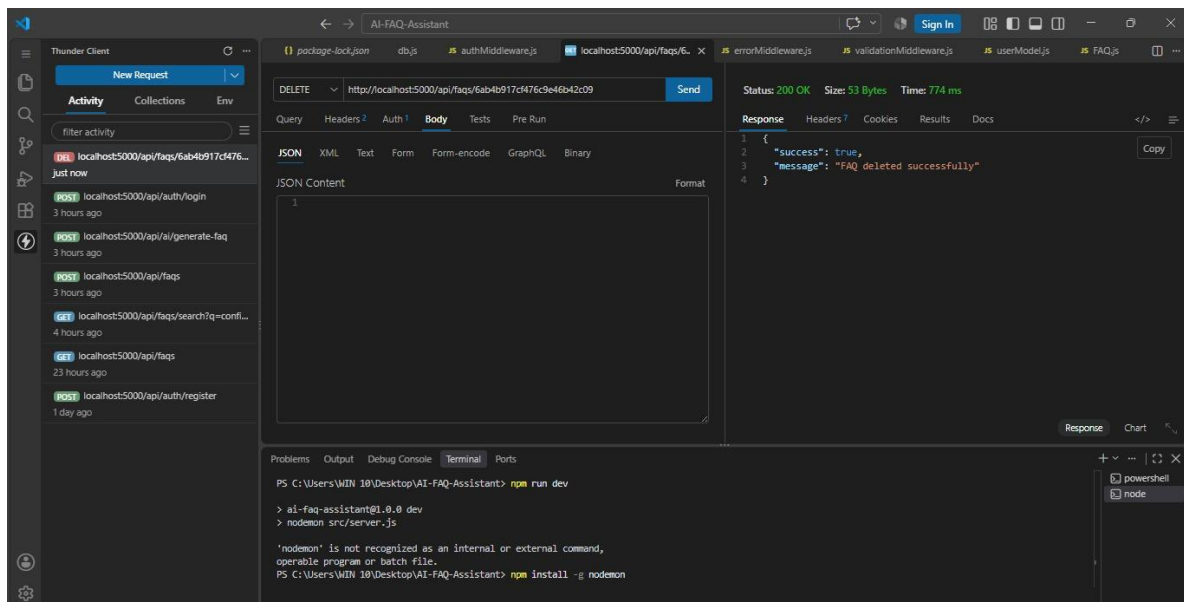
API testing evidence captured using Thunder Client.

Figure 6. FAQ List Retrieval – GET /api/faqs



API testing evidence captured using Thunder Client.

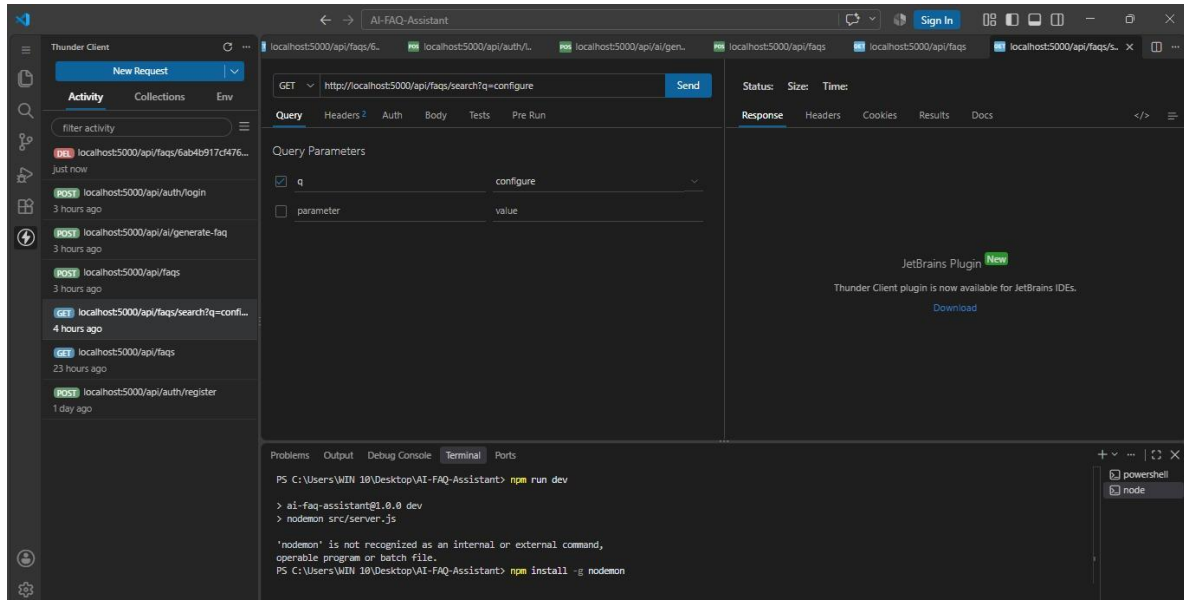
Figure 7. FAQ Deletion – DELETE /api/faqs/:id



API testing evidence captured using Thunder Client.

10. Test FAQ search.

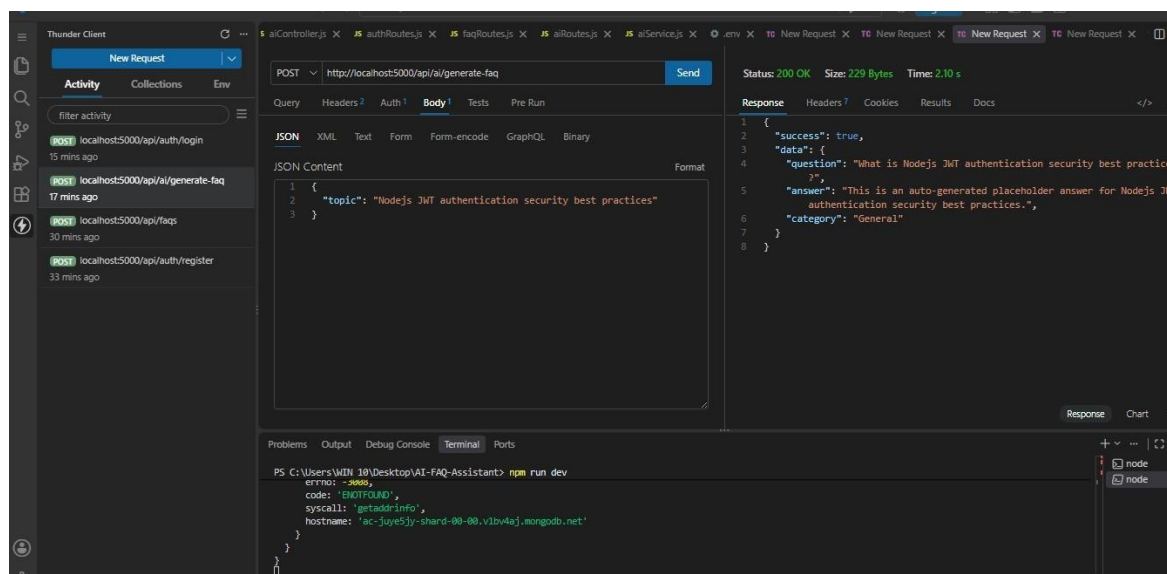
Figure 8. FAQ Search – GET /api/faqs/search?q=configure



API testing evidence captured using Thunder Client.

11. Test AI-based FAQ generation.

Figure 9. AI-Based FAQ Generation – POST /api/ai/generate-faq



API testing evidence captured using Thunder Client.

14. OUTPUT

The main API outputs include:

1. User Registration
2. User Login
3. FAQ Creation
4. FAQ Search
5. AI-generated FAQ

15. PROJECT DEMO

API Testing Video:

https://drive.google.com/file/d/1RD__JBdRWcW5vsgoYLnJOhWdG8NyuPEh/view?usp=drivesdk

Demo Link:

https://drive.google.com/file/d/1Fe8DR3DzddX_LdNXR80ux6eBghymsCHT/view?usp=drivesdk

